

Lec 18 Practical Exercise - 2

Lec 18 Practical Exercise - 2

Lecture	18 / 63 + intro
Duration	23:02
Captions	English (auto-generated)
Words	3,483
Video	sSDaKISydCA
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello all. Welcome to the hands-on series on fundamentals of generative AI and large language models, theory and practice. In today's session, we'll be implementing the different regularization techniques that can be used in autoencoders. Namely, the L1 regularization, L2 regularization, using dropouts, batch normalization technique, and early stopping technique. So, at the end of the session, we'll compare the different validation losses that we are implementing today and compare it whatever session that it was done earlier.

[00:39] So, initially, we import the necessary libraries. Importing the NumPy as NP, which is the numerical Python, which is required for dealing with the arrays or numbers. Then, importing the Matplotlib.pyplot as plt, which is required for data visualization. Then, we have importing of the TensorFlow, which is required to deal with the different deep learning models or the neural networks. From tensorflow.keras.datasets, we import the MNIST dataset. The MNIST dataset is a handwritten digit dataset, which has 10 different classes of numbers varying from 0 to 9. And then, we also import asterisk symbol. So, here asterisk means whatever functions which are available in TensorFlow as well as as well as Keras libraries, we can

[01:28] import once for all. We can just call those functions in the later part. So, that's why in short, we are importing it as asterisk function. And then, we import the models, which is required for building the neural network models. And then, we import the different regularizers, namely the L1, L2, and the various functions using which we can do the regularization, that is, dropouts, batch normalization, and the early stopping.

[01:55] So, once we import the necessary libraries, then comes to the loading of the data set. As mentioned earlier, we are making use of the MNIST data set, hence we are loading the data set and storing it in two different variables, training data set and the test data set. And then we try to convert it into a floating data type, which is very much helpful in the further computation of the neural networks. And then we normalize it. So, in order to perform normalization, we divide the input samples or the input pixel values by 255.

[02:30] So, this is done both on the training as well as the test data set. And then we try to reshape it in the shape of minus one and 784. Similarly, on the test data set minus one and 784. That means we are ignoring the label. So, in order to ignore the label, we can or in order to ignore the target variable as well, we can either make use of underscore or minus one. So, we are taking making use of only 784 different input neurons or input features.

[03:05] So, now coming to the first regularization, that is L1 regularization. So, input shape remains the same, which is 784. Then coming to defining the dense layer, which has one 128 neurons. Activation functions will be ReLU. And the regularization that technique that we are using is L1 regularization, once again of the order 10 to the power of minus four. So, in case of L1 regularization, we multiply the regularizer value, that is regularization value, into the absolute value of the weights. So, this will help in removing some of the neurons or giving less importance to different features. So, what we do is we give lot of importance only to the important features. So, it will help us in identifying only those

[03:54] features which are helpful in the model building or which are helpful in the interpretation of the data. So, regularization techniques will help in generalizing the models as well as it will helpful it is helpful in feature extraction, proper feature extraction. And then we have the dense layer which is at the output side, nothing but the decoding side. We have 784 different neurons activation function as the sigmoid activation function. And then we build the model based on both on the input and the output that we have defined. And then compiling of the model based on the optimizer and the loss function. So, we make use of the Adam optimizer and the binary cross entropy because we have already normalized the

[04:38] values between 0 and 1. So, we fit this regularized model on the training data set. Once again, it is multiple in number that is we consider it twice both for

the encoder as well as the decoder. Let the number of epochs be equal to five and batch size be equal to 256. So, as mentioned in the previous session, we can use either stochastic gradient descent or batch normalization or make use of different batch sizes. So, when we make use of different batch sizes, what happens is updation of the weights takes place at different batches. This is very much helpful in learning of the model.

[05:17] And now over validation data set is nothing but the X test and the X test once again both for the encoder as well as the decoder. So, let us see what is happening at each of the epochs. So, in epoch number one, the validation loss is 0.61. When moving to epoch number two, it has further reduced to 0.55, 0.50, 0.46 and 0.43. So, the loss has been reduced at different epochs. Now, let us compare this L1 regularization with the L2 regular regularization. So, L2 regularization, what has been done here is we try to multiply the regularization parameter, which is lambda, into squaring of the weights. We take the squaring of different weights. So, let us see this.

[06:18] So, in case of L1 regularization, the regularization parameter lambda will be multiplied with absolute values of the different weights. So, here WI refers to the weights which are measured at every batch. In case of L2 regularization, the same lambda or the regularization parameters is multiplied with the square of the weights. We can call it as AI or WI squared. Square of the weights. So, once again, WI refers to the weights here.

[07:01] Weights. So, WI, once again, weights at every batch. So, in both the cases, what we are trying to do is when we multiply this regularization parameter, which is a very finite number or a very small number, with the weights, the importance of the neurons, certain neurons will be lost. So, when we neuron or when we deactivate some of the neurons, which it means that we are deactivating the importance of different input features.

[07:35] So, our importance is given only to those features which are very much important, which is actually the feature extraction. So, coming back to the code here, uh L2 regularization, we define the input. Input shape is 784. We have the dense layer at the encoder. We have 128 neurons, and the activation function will be ReLU. And we make use of the regularizer, that is L2 regularization regularizer of the order 10 to the power of minus four. So, as mentioned in the previous session, we are free to use the regularizer of any value, maybe 10 to the power of minus one, minus two, minus three. So, it is based on how much we want to deactivate the neurons or how much less importance we are giving to a

[08:23] certain number of features or unimportant features. So, then next we define the dense layer. Once again, the dense layer required for the decoding part, 784 84 different neurons, activation function is the sigmoid activation function. Now, we build the model based on whatever input and the output that we have defined earlier,

and then we compile the model. So, during compilation, we make use of the optimizer as well as the different loss function. Optimizer is the Adam optimizer, and the loss function is binary cross entropy.

[08:57] Once the model has been built, we try to fit it on the training data set. Training data set you will be used to twice, one for the encoder, other one for the decoder. Let the number of epochs be equal to five, and the batch size as 256, and our validation data set is nothing but the tested data set, which is once again used twice for encoder and the decoder. So, at the end of every epoch, we are able to see that the validation loss is being decreased. So, at epoch one we had 0.15, epoch two it is reduced to 0.12. So, gradually we can see the fall in the validation loss.

[09:36] Now, when we try to compare this validation loss with the validation loss of the L1 regularization, in case of L1 regularization, the last epoch validation loss was 0.43. Now, here we have 0.09. So, which shows that when we make use of L2 regularization, that is when we square the weights and then multiply by the regularization parameter, the effect of neurons are very less. So, we are considering only the important features during model building. So, when we consider only the very much important features, it leads to a proper prediction. So, in our case of autoencoder, it leads in a proper reconstruction of the original image.

[10:22] So, L2 is better when compared to L1 regularization. Now, coming to the third type of regularization technique, which is the dropout, introducing some dropouts. So, dropouts means maybe at the hidden layer, we'll consider five different neurons. Hidden layer one, hidden layer two. So, what happens in the fully connected layer? First neuron is connected to every other neuron in the second hidden layer. Same with the second one is connected to the first one, the second one, the third one, fourth one I have already connected, the fifth one. The third one is also connected to the first one, the second one, the third one, the fourth and the fifth one. So, likewise, when we have connection throughout, it is a fully

[11:23] connected layer. Now, what happens is the first neuron, if I consider this as the first neuron and this as the second neuron. First neuron is always connected to the second neuron at every batch. So, it is trying to connect all the data points or all the samples or all the images. So, the association of neuron one and two is in such a way that every pattern is being recognized there. So, when we reconstruct the image, it will always give a proper or without blur uh the reconstructed image. So, what is a drawback here is when we come to the test data set. It is a unseen data. So, it will not be able to reconstruct it properly because we have not generalized the case. Now, with the help of dropout, what happens is let

[12:14] me take uh three neurons at a time. First neuron connected to the first neuron, first neuron connected to the second neuron, and the first neuron I will deactivate it from the third neuron. So, there is no connection between the first and the third

one. Maybe from the second one, I'll try to connect to the first one. Second one, let me avoid the connection to the second one. Let me connect the second one to the third one. So, we are dropping the certain connections between the neurons so that they will stop memorizing the values going from the first to the second, first to the third, and so on. Same thing to do with the third neuron.

[12:52] If I connect third to the first, let me avoid third to the second. Let me uh connect only third to the third. So, there is disconnection to the second neuron. So, memorizing of the values or memorizing of the patterns which are already observed is being lost here. So, this is a way to uh generalize the model or specification will be lost in this case. So, let us see how dropouts are being used here. So, dropout regularization, input features, input layer which are used to define the input shape that is 784 different input features.

[13:28] At the dense layer, I take 256 neurons, activation function being ReLU, but I introduce the dropouts. What kind of dropouts is I'm going to drop the 30% of the connections. So, out of 100%, I'm going to reduce the 30% of the connections. Now, at the decoding part, let us see dense layer 128, activation function ReLU. Again, I try to introduce some dropouts, which is once again 0.3 or 30% of the neurons will be lost here.

[14:01] Connections will be lost. Then, in the output layer, I once again have the dense layer 784 different neurons, activation function being sigmoid because it is the outermost layer or the output layer, and we have only two classes, that's why we consider the sigmoid activation function. So, now we build the model both on the input and the output being defined, compile the model, let the optimizer be Adam optimizer, loss function being binary cross entropy, and we try to fit this model, the new model, which is based on the dropouts on the X train as well as X train, epochs being equal to five, number of you know, batches, the batch size will be equal to 256, and the validation data being X test. So, when we see the number

[14:46] of epochs, we see the reduction in the validation loss. So, 0.14, 0.1, 0.1, 0.1, and 0.1042. So, we see that gradually there is a decrement in the in the loss value. So, this is a way to introduce dropouts or using dropouts, we can introduce the regularization in the neural networks. So, that was the third method. Now, coming to batch normalization. So, batch normalization means we try to normalize the values. So, how do we normalize the values?

[15:34] Normalized value, let me define using a variable x_{norm} , will be equal to the original value that is x , which is present in the sample or in the image, nothing but a pixel value, x minus the μ value divided by the standard deviation. So, when we try to do this, we are going to have most of the data which is normally distributed or we are going to have a Gaussian distribution like a bell-shaped curve. The values are centered around the zero or mean and standard deviation will be equal to one in our

case. So, with this, what happens is we try to scale the data or we try to normalize the data and get it in a proper range, maybe zero to one or minus one to one. So, even if we have a pixel value or intensity value like 255, it

[16:23] can be brought down to one. If we have a value which is negative, it can be brought down to zero. So, when we have in a particular range, again, we are trying to reduce the learning rate or the effort required for the model to learn the data set. So, this is one way of generalizing the data set. So, in order to perform batch normalization, we define the input layer with the shape 784. We define the dense layer with the 256 neurons. We try to have this batch normalization on the previous input and the activation function being ReLU. Same thing to do at the decoder part. We have the dense layer, 128 neurons, followed by the batch normalization and then the activation function being ReLU. And at

[17:13] the outmost layer, we have this dense 784 neurons. Once again, activation function being sigmoid. And then the model development and the compilation remains the same using Adam optimizer and the binary cross entropy. Once this is done, we are trying to fit the model batch normalized model on the training data set twice and then epochs being five, batch size being 256 and validation data being X test and the X test once again.

[17:44] So, when we have the different epochs, at different epochs we are seeing the variation in the losses. So, we can see that at the first epoch the noise itself is very or the loss itself is very much minimal, 0.13 and further it reduces to 0.08. So, which is the minimal loss that we have seen in the compared to all the other three cases. So, batch normalization has turned out to be a very good regularization technique compared to the above three techniques.

[18:18] Now, coming to the last regularization technique, which is the technique of early stopping. So, we have seen in the earlier cases like deep learning models, the technique of early stopping being used. So, early stopping means whenever we see that validation loss is not going to decrease or our model accuracy is not going to increase after a certain level, we clip that value. We are satisfied with that particular loss value or maybe the accuracy value. So, we clip that as the maximum value and further not go out with the more number of iterations or more number of epochs. So, when we do that, the number of computation or the number of operations being performed will be very minimal. So, this is one of

[19:02] the regularization techniques wherein we stop the model running at a particular stage when loss doesn't decrease or accuracy doesn't increase. So, threshold can be set by different users. We may limit the patience's level to two when there are no more changes after two epochs or maybe five epochs, we can stop the model development. So, how do we define this early stopping is? Early stop will be equal to uh from the Keras, we have this callbacks and we have this function called as early

stopping. So, early stopping works on the basis of three different arguments that is monitor.

[19:44] What we are going to monitor? Is it the accuracy or is it the loss? So, previously we have considered only the loss function. That's why let us monitor only the validation loss. And let the patience be equal to two. So, when we see that when the validation loss is not going to reduce after two different epochs, then we are going to stop the model training. And restore the best weights. That means in the back propagation, whenever we update the weights, we can you know, make use of those weights in the further front propagation or the forward propagation. So, we have to store those weights because those are the best possible weights or those are the weights which will help help in

[20:28] optimizing the model. So, what we do is we try to restore those best weights. So, how do you develop this early stopping is? Again, define the input and the shape of the input layer being 784, define both the encoder and the decoder. So, 256 further reduce to 128 and then we have the 784. Activation function remains the same which is ReLU and the sigmoid at the outermost layer. And then we have the model which is built both on the input as well as the output. We compile the same model using the Adam optimizer and the binary cross entropy. And then try to fit this on the training data X train. Once again, the X train both for the encoder and the decoder. So, when we do this, we see at different epochs, we are going to see

[21:20] the reduction in the loss values. So, first we have seen a 0.11, then being reduced to 0.09, 0.08, 0.0799, and 0.0722. So, we in this case of high epochs, we are not able to see the early stopping. So, if I had run this model for maybe 10 epochs or 50 or 20 epochs, maybe the value would have been saturated. So, saturation means suppose I take epoch number four, if the value is 0.0799, even for epoch number five, if I get 0.0799, and epoch number six, 0.0799, after two or three epochs, if the value remains the same, then the model is going to stop the training. So, this is the best possible validation loss which is achieved. So, the model is going to stop the training. So, that is called as early stopping.

[22:17] So, in this session, we have seen the different types of regularization techniques which could be used in the autoencoder. So, first we started with the L1 regularization, then we moved to L2 regularization. When we compared, we saw that L2 regularization performed the best when compared to L1 because there the weights are being multiplied are being squared. And we saw the third case, wherein we have seen the early stopping, we have seen the batch normalization also. So, when we compare all the five different techniques, we see that the early stopping has worked the best for us.

[22:55] Thank you.